

Simulation Method for Autonomous Vehicle Perception and Control Using Geometric Wave-guide Masking and Perspective Projection

Abstract

Testing and validating autonomous vehicle safety requires extensive driving data, but real-world testing in adverse weather conditions incurs high costs and significant risks. While existing high-fidelity 3D simulators (e.g., CARLA) provide realistic rendering, they demand massive computational resources, making them difficult to run on lightweight devices. Conversely, simple distance-calculation simulators fail to accurately model how physical waves scatter and reflect off road environments. This paper proposes a lightweight simulation methodology utilizing the 2D wave equation to generate virtual radar/sonar signals. By employing geometric wave-guide masking based on road topography and applying perspective projection from the driver's viewpoint, the system effectively tests perception robustness in harsh weather. The proposed method reduces computational complexity to $O(N)$, enabling real-time execution at over 60 FPS on embedded Application Processors (APs), while simultaneously offering a practical testbed for signal attenuation and phantom obstacle phenomena.

1. Introduction

The safety of autonomous driving systems is heavily dependent on the robustness of their sensor perception and control algorithms, particularly under adverse weather conditions. Rain, snow, and fog introduce severe sensor noise and signal attenuation, which can lead to critical malfunctions. To address this, we introduce a novel simulation framework designed to physically model wave propagation while maintaining extreme computational efficiency.

This methodology bridges the gap between physical reality (Bird's Eye View) and cognitive perception (Perspective View) by mathematically isolating and synchronizing the two spaces. By doing so, it serves as a highly efficient Proof of Concept (PoC) model for industry applications, potentially reducing real-world road testing costs by over 90%.

2. Methodology

2.1. Wave Propagation Engine

The core of the simulation relies on a discretized wave equation to replicate radar and sonar propagation with low computational overhead. The wave engine utilizes a 5-point stencil Laplacian operation to generate waves:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u$$

Within a discretized grid space (e.g., 80×200 resolution), the engine periodically emits impulses from the front of the ego vehicle.

2.2. Dynamic Wave-guide Masking

To simulate signal directivity, the geometric shape of the road acts as a mask, focusing wave energy toward a forward vanishing point to create a “virtual wave-guide” effect. The road geometry is defined by a sine function:

$$x = A \sin(By)$$

- A represents the lateral deviation width.
- B defines the curvature change rate per driving distance.

A dynamic mask (`road_mask`) forces wave intensities outside the specified road boundaries to terminate, guiding the waves strictly along the curved path of the road. This allows the simulation of various scenarios, including S-curves, intersections, and merging roads.

2.3. Adverse Weather Noise Modeling

The simulator introduces physical alterations to the wave dynamics to replicate specific weather conditions without requiring actual sensor hardware:

- **Rain (Signal Attenuation):** Modifies the damping rate of the wave to simulate energy loss.
- **Snow (Phantom Obstacles):** Injects Gaussian noise and generates random particles within the grid to replicate scattering and false-positive “phantom” obstacles.

2.4. Perspective Projection Mapping

To evaluate the discrepancy between physical detection and human/camera-like visual cognition, physical coordinates (x, y) are non-linearly projected into perspective coordinates (x', y') :

$$x' = V_x + (x - V_x) \cdot \frac{K}{\text{dist} + C}$$
$$y' = f(\text{dist}^{0.4})$$

- K serves as the conversion factor between physical distance and screen pixels.
- C acts as a perspective offset for depth rendering.

2.5. Real-Time Control Feedback Loop

Moving beyond simple distance-threshold controls, the system integrates a real-time feedback loop combining Physics, Perception, and Control. When the average amplitude (pressure) of waves reflected from obstacles and received in the sensor region exceeds a specific threshold, the vehicle generates a smooth braking force utilizing an Exponential Moving Average (EMA).

3. Python Implementation

The following code demonstrates the simulation framework built utilizing `numpy` for matrix operations and `matplotlib` for visualization, encapsulating the physics engine, masking logic, and control loops.

Python

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from matplotlib.patches import Rectangle
from IPython.display import HTML

# =====
# 1. Configuration & Engine
# =====
WIDTH, HEIGHT = 80, 200 # [cite: 66]

# Class structure introduced to manage independent simulation instances [cite: 67]
class Simulation:
    def __init__(self, ax, weather_mode, title): # [cite: 69]
        self.ax = ax
```

```

self.weather_mode = weather_mode
self.title = title
self.init_weather_params()
self.init_state()
self.init_visuals()

def init_weather_params(self): # [cite: 76]
    if self.weather_mode == 1: # Rain (Signal Attenuation) [cite: 77]
        self.damping = 0.94 # [cite: 78]
        self.noise_level = 0.5 # [cite: 79]
        self.label = "RAIN (Attenuation)" # [cite: 80]
    elif self.weather_mode == 2: # Snow (Scattering Noise & Phantom Obstacles) [cite:
        self.damping = 0.98 # [cite: 82]
        self.noise_level = 1.2 # [cite: 83]
        self.label = "SNOW (Phantom & Noise)" # [cite: 84]
    else: # Sunny (Clear) [cite: 85]
        self.damping = 0.985 # [cite: 86]
        self.noise_level = 0.2 # [cite: 87]
        self.label = "SUNNY (Clear)" # [cite: 88]

def init_state(self): # [cite: 89]
    self.current_wave = np.zeros((HEIGHT, WIDTH)) # [cite: 90]
    self.previous_wave = np.zeros((HEIGHT, WIDTH)) # [cite: 91]
    self.road_mask = np.zeros((HEIGHT, WIDTH)) # [cite: 92]
    self.road_center_x = np.zeros(HEIGHT) # [cite: 93]

    # Road Generation (Sine Wave) [cite: 94]
    for y in range(HEIGHT): # [cite: 95]
        center_x = WIDTH // 2 + np.sin(y * 0.03) * 18 # [cite: 96]
        self.road_center_x[y] = center_x # [cite: 97]
        self.road_mask[y, int(center_x-14):int(center_x+14)] = 1.0 # [cite: 98]

    self.ego_y, self.ego_speed = 20, 3.5 # [cite: 99]
    self.target_y, self.target_speed = 170.0, 1.2 # [cite: 100]
    self.lane_offset, self.frame_count, self.stop_timer = 0, 0, 0 # [cite: 101]

def init_visuals(self): # [cite: 102]
    self.ax.set_title(self.title) # [cite: 103]
    self.ax.set_xlim(0, WIDTH); self.ax.set_ylim(0, HEIGHT) # [cite: 104]
    self.ax.set_facecolor('#222222') # [cite: 105]

    # Wave Visualization (Background) [cite: 106]
    self.im_wave = self.ax.imshow(self.current_wave, cmap='RdBu', vmin=-10, vmax=10,

    # Vehicle Visualization [cite: 108]
    self.ego_viz = Rectangle((0,0), 6, 8, color='#00BFFF', label='Ego') # [cite: 109]
    self.target_viz = Rectangle((0,0), 6, 8, color='#FF4500', label='Target') # [cite

```

```

self.ax.add_patch(self.ego_viz); self.ax.add_patch(self.target_viz) # [cite: 111]

# Status Text UI [cite: 112]
self.status_text = self.ax.text(0.05, 0.95, "", transform=self.ax.transAxes, # [c
                                fontsize=9, color='white', verticalalignment='top
                                bbox=dict(boxstyle="round,pad=0.3", fc="black", a

def update_wave(self): # [cite: 116]
    # 5-point stencil Laplacian computation
    laplacian = (np.roll(self.current_wave, 1, axis=0) + np.roll(self.current_wave, -
                        np.roll(self.current_wave, 1, axis=1) + np.roll(self.current_wave, -
                        4 * self.current_wave) # [cite: 119]

    new_wave = 2 * self.current_wave - self.previous_wave + 0.5 * laplacian # [cite:
    new_wave *= self.damping # Apply weather damping [cite: 121]
    new_wave *= self.road_mask # Apply waveguide mask [cite: 122]

    # [Adverse Weather Effect: Snow] Generate phantom obstacles [cite: 123]
    if self.weather_mode == 2 and self.frame_count % 5 == 0: # [cite: 124]
        ry = np.random.randint(self.ego_y+20, HEIGHT-10) # [cite: 125]
        rx = np.random.randint(WIDTH//2-10, WIDTH//2+10) # [cite: 126]
        new_wave[ry-2:ry+2, rx-2:rx+2] += np.random.uniform(5, 15) # [cite: 127]

    # Vehicle reflection handling [cite: 128]
    safe_ty = int(np.clip(self.target_y, 0, HEIGHT-1)) # [cite: 129]
    tx = int(self.road_center_x[safe_ty]) # [cite: 130]
    new_wave[safe_ty-4:safe_ty+4, tx-3:tx+3] = 0 # [cite: 131]

    ex = int(self.road_center_x[self.ego_y]) # [cite: 132]
    new_wave[self.ego_y-2:self.ego_y+5, ex-3:ex+3] = 0 # [cite: 133]

    self.previous_wave[:] = self.current_wave[:] # [cite: 134]
    self.current_wave[:] = new_wave[:] # [cite: 135]

def control_ego_car(self): # [cite: 136]
    ex = int(self.road_center_x[self.ego_y]) # [cite: 137]
    sensor_region = self.current_wave[self.ego_y+15:self.ego_y+35, ex-8:ex+8] # [cite

    # [Adverse Weather Effect: Noise Injection] [cite: 139]
    pressure = np.mean(np.abs(sensor_region)) + np.random.normal(0, self.noise_level)
    pressure = max(0, pressure) # [cite: 141]

    # EMA based braking logic
    if pressure > 5.0: # [cite: 142]
        target_v, response = 0.0, 0.25; self.stop_timer += 1 # [cite: 143]
    elif pressure > 1.5: # [cite: 144]
        target_v, response = self.target_speed, 0.06; self.stop_timer += 1 # [cite: 1

```

```

else: # [cite: 146]
    if self.stop_timer > 40 and pressure < 0.5: # [cite: 147]
        target_v, response = 4.0, 0.04; self.ego_speed > 2.0 and setattr(self, 's
    else: # [cite: 149]
        target_v, response = 4.0, 0.02 # [cite: 150]

self.ego_speed += (target_v - self.ego_speed) * response # [cite: 151]
if self.ego_speed < 0: self.ego_speed = 0 # [cite: 152]
return pressure # [cite: 153]

def update_scenario(self): # [cite: 154]
    self.lane_offset = (self.lane_offset - self.ego_speed * 2.0) % 40 # [cite: 155]
    if self.frame_count > 150: # [cite: 156]
        self.target_speed += 0.03 # [cite: 157]
        if self.target_speed > 3.0: self.target_speed = 3.0 # [cite: 158]

    self.target_y += (self.target_speed - self.ego_speed) # [cite: 159]
    self.target_y = np.clip(self.target_y, self.ego_y + 9, HEIGHT + 50) # [cite: 160]

def step(self): # [cite: 161]
    self.frame_count += 1 # [cite: 162]
    if self.frame_count % 6 == 0: # Emit pulse [cite: 163]
        ex = int(self.road_center_x[self.ego_y]) # [cite: 164]
        self.current_wave[self.ego_y+6, ex-8:ex+8] = 35.0 # [cite: 165]

    self.update_wave() # [cite: 166]
    self.update_scenario() # [cite: 167]
    pressure = self.control_ego_car() # [cite: 168]

    # Update Visualizations [cite: 169]
    self.im_wave.set_data(self.current_wave) # [cite: 170]
    safe_ty = int(np.clip(self.target_y, 0, HEIGHT-1)) # [cite: 171]
    tx = int(self.road_center_x[safe_ty]) # [cite: 172]
    ex = int(self.road_center_x[self.ego_y]) # [cite: 173]

    self.ego_viz.set_xy((ex-3, self.ego_y-4)) # [cite: 174]
    self.target_viz.set_xy((tx-3, self.target_y-4)) # [cite: 175]

    bumper_gap = (self.target_y - 4) - (self.ego_y + 4) # [cite: 176]
    status = "BRAKING" if self.ego_speed < self.target_speed + 0.5 else "CHASING" # [
    self.status_text.set_text(f"[{self.label}]\nSpeed: {self.ego_speed:.2f}\nGap: {bu

    # Return updated artists for animation [cite: 179]
    return [self.im_wave, self.ego_viz, self.target_viz, self.status_text] # [cite: 1

```

```

# =====

```

```

# 2. Main Execution

```

```
# =====  
fig, axes = plt.subplots(1, 3, figsize=(18, 8)) # [cite: 184]  
fig.patch.set_facecolor('#111111') # Set global background color [cite: 185]  
  
# Create three independent simulation instances [cite: 186]  
sims = [ # [cite: 187]  
    Simulation(axes[0], weather_mode=0, title="1. SUNNY (Clear)", # [cite: 188]  
    Simulation(axes[1], weather_mode=1, title="2. RAIN (High Attenuation)", # [cite: 189]  
    Simulation(axes[2], weather_mode=2, title="3. SNOW (High Noise & Phantom)") # [cite:  
] # [cite: 191]  
  
# Master update function for animation [cite: 192]  
def master_update(frame): # [cite: 193]  
    all_artists = [] # [cite: 194]  
    for sim in sims: # [cite: 195]  
        all_artists.extend(sim.step()) # [cite: 196]  
    return all_artists # [cite: 197]  
  
# Generate Animation [cite: 198]  
# Note: If screen renders blank, try setting blit=False [cite: 199]  
ani = FuncAnimation(fig, master_update, frames=250, interval=40, blit=True) # [cite: 200]  
plt.close() # [cite: 201]  
HTML(ani.to_jshtml()) # [cite: 202]
```

4. Conclusion

This paper detailed a computationally efficient methodology for testing autonomous vehicle perception and control. By substituting high-overhead 3D ray-tracing with discretized wave equations and dynamic road masks, the system allows for the rapid iteration and validation of control algorithms. Furthermore, the programmatic injection of weather-based noise variables enables rigorous edge-case testing, making this methodology a highly viable solution for embedded systems and industrial proof-of-concepts.

5. Prior Art and Related Literature

The following prior patent literature provides foundational context for the simulation of autonomous driving environments and the modeling of sensor behavior under adverse weather conditions:

- **Patent Literature 1:** Republic of Korea Patent Registration No. 10-1890251 (Published: August 16, 2018), "Virtual driving environment simulation system for autonomous vehicle and method thereof".
- **Patent Literature 2:** Republic of Korea Patent Registration No. 10-2143521 (Published: August 3, 2020), "Method and apparatus for simulating severe weather condition of radar sensor for autonomous vehicle".